

COMS 3110 Introduction to the Design and Analysis of Algorithms, Homework 5

Mobina Amrollahi

Due: December 6th, 11:59pm

Late submission policy. An assignment that is submitted one day late will get a penalty of 10%. An assignment that is submitted two days late will get a penalty of 20%. Assignments submitted after two days will not be graded and will get no points. For example, if an assignment is due on Friday by midnight, a submission on Saturday will be penalized by 10%, and a submission on Sunday will be penalized by 20%. A submission on Monday will get no points.

Submission format. Homework solutions will have to be typed. You can use word, LaTeX, or any other type-setting tool to type your solution. Your submission file should be in pdf format. Do **NOT** submit a photocopy of handwritten homework except for diagrams that can be hand-drawn and scanned. We reserve the right **NOT** to grade homework that does not follow the formatting requirements. Name your submission file: `<Your-net-id>-3110-hw5.pdf`. For instance, if your netid is `asterix`, then your submission file will be named `asterix-3110-hw5.pdf`. Each student must hand in their own assignment. If you discussed the homework or solutions with others, a list of collaborators must be included with each submission. Each of the collaborators has to write the solutions in their own words (copies are not allowed).

General Requirements

- When proofs are required, do your best to make them both clear and rigorous. Even when proofs are not required, you should justify your answers and explain your work.
- When asked to present a construction, you should show the correctness of the construction.

Some Useful (in)equalities

- $\sum_{i=1}^n i = \frac{n(n+1)}{2}$

- $\sum_{i=1}^n i^2 = \frac{n(n+1)(2n+1)}{6}$
 - $2^{\log_2 n} = n$, $a^{\log_b n} = n^{\log_b a}$, $n^{n/2} \leq n! \leq n^n$, $\log x^a = a \log x$
 - $\log(a \times b) = \log a + \log b$, $\log(a/b) = \log a - \log b$
 - $a + ar + ar^2 + \dots + ar^{n-1} = \frac{a(r^n-1)}{r-1}$
 - $1 + \frac{1}{2} + \frac{1}{2^2} + \dots + \frac{1}{2^n} = 2(1 - \frac{1}{2^{n+1}})$
 - $1 + 2 + 4 + \dots + 2^n = 2^{n+1} - 1$
-

1. (10 pts) You are helping a group of ethnographers analyze some oral history data. The ethnographers interviewed members of a village and learned about the birth and death a set of n people: $P_1, P_2, \dots, P_n$. The information is categorized as follows:

- For some i and j , person P_i died before person P_j was born
- For some i and j , the life spans of P_i and P_j overlapped at least partially.

As these are word of mouth information, it becomes important to validate the data. You are tasked with writing an algorithm which takes as input the learned information (about relative ordering of birth and death of n people) and verify its validity.

Answer

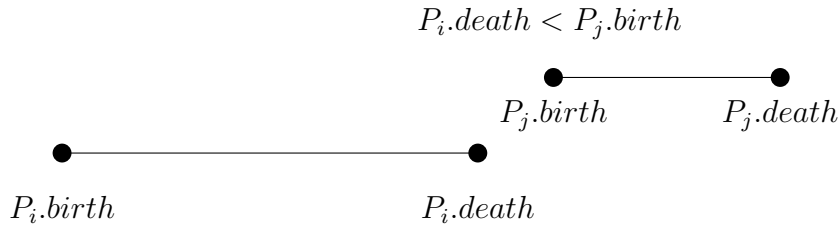
Description: Suppose we call the corresponding information by their associated numbers:

- (1) For some i and j , person P_i died before person P_j was born.
- (2) For some i and j , the life spans of P_i and P_j overlapped at least partially.

To check the validity of information, we need to check that the learned information from type (1) and type (2) don't contradict each other and are internally consistent.

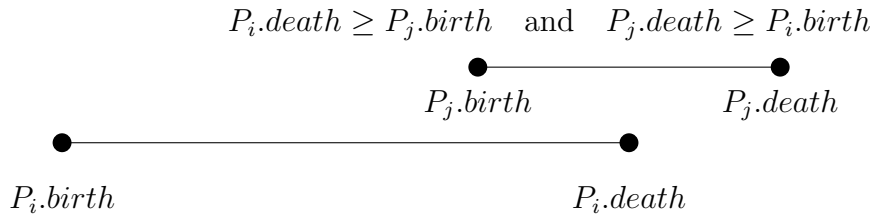
Let $P.birth$ denote the time a person P is born, and $P.death$ denote the time the person is dead. Hence, the two types of information can be translated to:

- (1) For some i and j , person P_i died before person P_j was born:



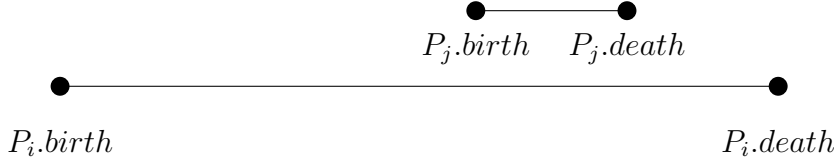
- (2) For some i and j , the life spans of P_i and P_j overlapped at least partially. This means that the lifespan of P_i and P_j cannot be disjoint in terms of time. Whether one of them died after the other was born, or whether the lifespan of one, from beginning to end, happened during the lifespan of the other. Hence, one of the following four possibilities might occur:

1.



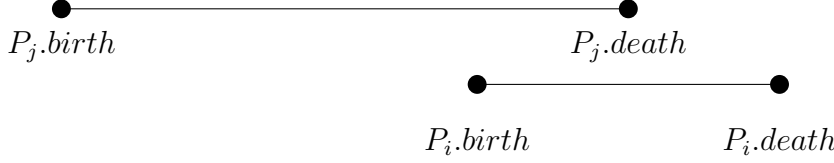
2.

$$P_i.death \geq P_j.birth \quad \text{and} \quad P_j.death \geq P_i.birth$$



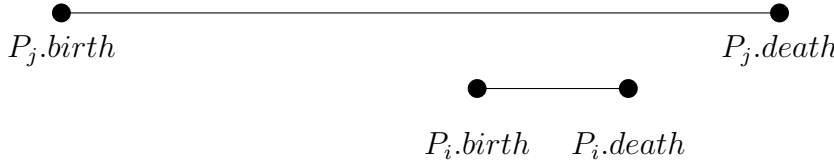
3.

$$P_i.death \geq P_j.birth \quad \text{and} \quad P_j.death \geq P_i.birth$$



4.

$$P_i.death \geq P_j.birth \quad \text{and} \quad P_j.death \geq P_i.birth$$



Notice that, in all four possibilities $P_i.death \geq P_j.birth$ and $P_j.death \geq P_i.birth$, which means it's impossible for the same i and j for both types of information to work. The first information indicates that i has never shared the same timeline of its lifespan with j ; meanwhile, the second information says the opposite. Hence, for any two people, i and j , the validity of the information means at most one of them works.

Since the given information is mixed, we will start by dividing it into two types: **Type1** = $\square$ and **Type2** = $\{\}$. While constructing the adjacency list for **Type1**, we will also construct a directed graph $\mathcal{G} = (V, E)$, where V represents the set of n people: $P_1, P_2, \dots, P_n$, and E represents the dependency relationship between two people. Specifically, the edge (P_i, P_j) indicates that person P_i died before person P_j was born. Once the information is divided into the two types, to check the validity of the information, we need to verify two things in $\mathcal{G} = (V, E)$:

1. **Checking the Validity of information Type (1):** $\mathcal{G} = (V, E)$ should be acyclic. Otherwise, suppose there exists a cycle C in $\mathcal{G} = (V, E)$, and $C = P_k \rightarrow P_{k+1} \rightarrow P_{k+2} \rightarrow \dots \rightarrow P_{k+T} \rightarrow P_k$. Based on the dependency relationship, we

can say:

$$\begin{aligned}
P_k.death &< P_{k+1}.birth \\
P_{k+1}.death &< P_{k+2}.birth \\
P_{k+2}.death &< P_{k+3}.birth \\
&\vdots \\
P_{k+T-1}.death &< P_{k+T}.birth \\
P_{k+T}.death &< P_k.birth
\end{aligned}$$

Since for any person P , $P.birth < P.death$, hence:

$$\begin{aligned}
P_k.death &< P_{k+1}.birth < P_{k+1}.death < P_{k+2}.birth < P_{k+2}.death < P_{k+3}.birth < \\
&\cdots < P_{k+T-1}.death < P_{k+T}.birth < P_{k+T}.death < P_k.birth
\end{aligned}$$

Which means:

$$P_k.death < P_k.birth$$

Which cannot be true. Therefore, for the information to be valid, $\mathcal{G} = (V, E)$ must be acyclic. Since $\mathcal{G} = (V, E)$ is directed, if it's acyclic, a topological ordering must exist. If not, the graph has a cycle, and the information is not valid. Hence, in the algorithm, after constructing $\mathcal{G} = (V, E)$ with respect to information type (1), we will check if there exists a topological ordering or not. If yes, we will proceed to check the validity of information type (2), and if not, we will return **False**, indicating that the information is not valid.

2. **Checking the Validity of information Type (2):** If for some i and j , information type (2) is received, in $\mathcal{G} = (V, E)$, there shouldn't be a path from P_i to P_j or from P_j to P_i . Otherwise, assume there's a path P from P_i to P_j , $P = P_i \rightarrow P_{i+1} \rightarrow P_{i+2} \rightarrow \cdots \rightarrow P_{i+T} \rightarrow P_j$, then based on the dependency relationship, we can say:

$$\begin{aligned}
P_i.death &< P_{i+1}.birth \\
P_{i+1}.death &< P_{i+2}.birth \\
P_{i+2}.death &< P_{i+3}.birth \\
&\vdots \\
P_{i+T-1}.death &< P_{i+T}.birth \\
P_{i+T}.death &< P_j.birth
\end{aligned}$$

Since for any person P , $P.birth < P.death$, hence:

$$\begin{aligned}
P_i.death &< P_{i+1}.birth < P_{i+1}.death < P_{i+2}.birth < P_{i+2}.death < P_{i+3}.birth < \\
&\cdots < P_{i+T-1}.death < P_{i+T}.birth < P_j.death
\end{aligned}$$

Which means:

$$P_i.death < P_j.birth$$

However, based on information type (2) we know:

$$P_i.death \geq P_j.birth \quad \text{and} \quad P_j.death \geq P_i.birth$$

Which cannot be true. The same can be shown for not having a path between P_j to P_i . Therefore, for information type (2), there shouldn't be a path between the corresponding vertices in $\mathcal{G} = (V, E)$. Hence, after constructing $\mathcal{G} = (V, E)$ with respect to information type (1), and checking if there exists a topological ordering, for any piece of information type (2), we will run a DFS algorithm to check whether there's a path between the corresponding vertices. If yes, we will return **False**, otherwise, we will return **True**.

Pseudo code:

```
INPUT P = {P1, P2, ..., Pn}, info = [info1, info2, info3, ..., infom]
Initialization:
// Graph representation, adjacency list
G = {i: [] for i in range(n)}
// List for Type1 information
Type1 = []
// Hash table for Type2 information
Type2 = {}

// Deviding the information, Constructing the graph
for i in range(info):
    if info[i] is type 1:
        G[i].append(j)
        Type1[i].append(j)
    elif info[i] is type 2:
        Type2[i] = j
        Type2[j] = i //Since both are sharing their lifespan

// Check the validity of type 1 information
// Which is equivalent to checking whether there's a cylce using DFS
for all vertex v in G:
    v.explored = false

for all vertex v in G:
    v.rec_stack = false

// Perform DFS for every node that is not yet visited
```

```

for i in range(V):
    if v.explored == false and Cycle(G, v):
        // Cycle detected and information invalid
        return False

Cycle(G, v):
for all vertex v in G:
    if v.explored == false:
        // Mark the node as explored
        v.explored = True
        // Mark the node as part of the recursion stack
        v.rec_stack = True

        for all neighbors w of v:
            if w.explored == False and Cycle(G, w):
                return False // Cycle detected
            elif w.rec_stack == True:
                return False // Cycle detected

        // After visiting all neighbors,
        remove the node from the recursion stack
        v.rec_stack = False

// Check the validity of Type2 information
for each key in Type2:
    for each value in Type2[key]:
        DFS(key, value, G)

DFS(v, target, G):
    # Mark the node as explored
    v.explored = True

    # If the current node is the target, return True (path found)
    if v == target:
        return False // a path found

    # Recur for all neighbors of v
    for all neighbors w of v:
        if not w.explored: // If the neighbor hasn't been explored
            if DFS(w, target, graph):
                return False // a path found

```

```
return True // no path found
```

2. (10 pts) Solve the following recurrence equations using the recurrence tree method. Show the table as done in the lecture.

(a) (5 pts)

$$T(n) = \begin{cases} 1 & \text{if } n < 2 \\ 4 \cdot T(\frac{n}{2}) + n & \text{if } n \geq 2 \end{cases}$$

Answer

Level No.	Problem Size	No. of Nodes	Work per Node	Total Work
1	$\frac{n}{2^0}$	1	n	$1 \times n = n$
2	$\frac{n}{2^1}$	4^1	$\frac{n}{2^1}$	$4 \times \frac{n}{2^1} = 2^1 n$
3	$\frac{n}{2^2}$	4^2	$\frac{n}{2^2}$	$4^2 \times \frac{n}{2^2} = 2^2 n$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
i	$\frac{n}{2^{i-1}}$	4^{i-1}	$\frac{n}{2^{i-1}}$	$4^{i-1} \times \frac{n}{2^{i-1}} = 2^{i-1} n$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
$l-1$	$\frac{n}{2^{l-2}}$	4^{l-2}	$\frac{n}{2^{l-2}}$	$4^{l-2} \times \frac{n}{2^{l-2}} = 2^{l-2} n$

Suppose at level l the problem size = 1, then

$$\begin{aligned} \frac{n}{2^{l-1}} = 1 &\Rightarrow n = 2^{l-1} \\ &\Rightarrow \log(n) = l - 1 \\ &\Rightarrow l = \log(n) + 1 \end{aligned}$$

So for all i from 1 to $\log(n)$, the work done at level i is $2^{i-1}n$. At level $\log(n)+1$, the work done at each node is 1, and there are $4^{\log(n)+1-1} = 4^{\log(n)} = 2^{\log(n)} \times 2^{\log(n)} = n^2$ nodes, so total work at level $\log(n) + 1$ is n^2 .

Hence, the total work done is:

$$\begin{aligned}
\text{Total Work} &= \text{Total work until level } l-1 + \text{Work done at level } l \\
&= \underbrace{n + 2^1n + 2^2n + \cdots + 2^{i-1}n + \cdots + 2^{(l-1)-1}n}_{\text{Work until level } l-1} + \underbrace{n^2}_{\text{Work at Level } l} \\
&= \sum_{i=1}^{l-2} 2^{i-1}n + n^2 \\
&= n \sum_{i=1}^{l-2} 2^{i-1} + n^2 \\
&= n \times (2^{l-1} - 1) + n^2 \quad (\text{Replacing } l-1 \text{ with } \log(n)) \\
&= n \times (2^{\log(n)} - 1) + n^2 \\
&= n \times (n - 1) + n^2 \\
&= 2n^2 - n \in O(n^2)
\end{aligned}$$

(b) (5 pts)

$$T(n) = \begin{cases} 1 & \text{if } n < 3 \\ T(\frac{n}{3}) + n & \text{if } n \geq 3 \end{cases}$$

Answer

Level No.	Problem Size	No. of Nodes	Work per Node	Total Work
1	$\frac{n}{3^0}$	1	n	$1 \times n = n$
2	$\frac{n}{3^1}$	1	$\frac{n}{3^1}$	$1 \times \frac{n}{3^1} = \frac{n}{3^1}$
3	$\frac{n}{3^2}$	1	$\frac{n}{3^2}$	$1 \times \frac{n}{3^2} = \frac{n}{3^2}$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
i	$\frac{n}{3^{i-1}}$	1	$\frac{n}{3^{i-1}}$	$1 \times \frac{n}{3^{i-1}} = \frac{n}{3^{i-1}}$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
$l-1$	$\frac{n}{3^{l-2}}$	1	$\frac{n}{3^{l-2}}$	$1 \times \frac{n}{3^{l-2}} = \frac{n}{3^{l-2}}$
l	$\frac{n}{3^{l-1}} = 2$	1	1	$1 \times 1 = 1$

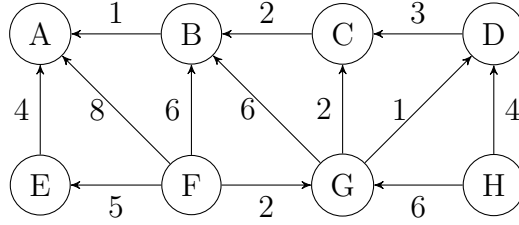
For the last row:

$$\begin{aligned}
\frac{n}{3^{l-1}} = 2 &\Rightarrow n = 2 \times 3^{l-1} \\
&\Rightarrow \frac{n}{2} = 3^{l-1} \\
&\Rightarrow \log_3\left(\frac{n}{2}\right) = l-1 \\
&\Rightarrow l = \log_3\left(\frac{n}{2}\right) + 1
\end{aligned}$$

Hence, the total work done is:

$$\begin{aligned}
\text{Total Work} &= \text{Total work until level } l-1 + \text{Work done at level } l \\
&= \underbrace{\frac{n}{3^0} + \frac{n}{3^1} + \frac{n}{3^2} + \cdots + \frac{n}{3^{l-1}} + \cdots + \frac{n}{3^{l-2}}}_{\text{Total work until level } l-1} + \underbrace{1}_{\text{Work at Level } l} \\
&= \sum_{i=1}^{l-2} \frac{n}{3^{i-1}} + 1 \\
&= n \sum_{i=1}^{l-2} \frac{1}{3^{i-1}} + 1 \\
&= n \times \frac{\frac{1}{3^{l-1}} - 1}{\frac{1}{3} - 1} + 1 \\
&= n \times \frac{\frac{1-3^{l-1}}{3^{l-1}}}{\frac{1-3}{3}} + 1 \\
&= n \times \frac{\frac{3^{l-1}-1}{3^{l-1}}}{\frac{3-1}{3}} + 1 \\
&= n \times \frac{3 \times (3^{l-1} - 1)}{2 \times (3^{l-1})} + 1 \quad (\text{Replacing } l-1 \text{ with } \log_3(\frac{n}{2})) \\
&= n \times \frac{3 \times (\frac{n}{2} - 1)}{2 \times (\frac{n}{2})} + 1 \\
&= n \times \frac{\frac{3n}{2} - 3}{n} + 1 \\
&= \frac{3n}{2} - 3 + 1 \\
&= \frac{3n}{2} - 2 \in O(n)
\end{aligned}$$

3. **(10 pts)** Recall the Dijkstra's algorithm as presented in class. Consider the following weighted graph and write the valuation of d -value for each vertex at the end of each iteration of while-loop (that iterates until the priority queue is empty) in the algorithm. The source vertex is H.



(Add more rows to the following table, if needed, for your solution.)

Answer We will be adding two new rows to the table: "d-value," which will represent the final d-value for each vertex, and "Parent," which will represent the final parent of each vertex. In addition, we will define set S , which initially holds the source vertex "H," and we will gradually add other vertices to it with respect to Dijkstra's algorithm. You may see numbers with powers as letters in the table entries, such as 4^H . The H represents the parent of the vertex after that iteration. Furthermore, once a vertex is added to S , we will be circling around its d-value at the iteration it's added to S , which indicates that would be the final d-value of that vertex. Finally, since we have eight vertices with all the edges having positive values, we will be iterating eight times.

Initial d-value	∞	∞	∞	∞	∞	∞	∞	0
Iteration	A	B	C	D	E	F	G	H
1	∞	∞	∞	4^H	∞	∞	6^H	0
2	∞	∞	7^D	4^H	∞	∞	6^H	0
3	∞	12^G	7^D	4^H	∞	∞	6^H	0
4	∞	9^C	7^D	4^H	∞	∞	6^H	0
5	10^B	9^C	7^D	4^H	∞	∞	6^H	0
6	10^B	9^C	7^D	4^H	∞	∞	6^H	0
7	10^B	9^C	7^D	4^H	∞	∞	6^H	0
8	10^B	9^C	7^D	4^H	∞	∞	6^H	0
d-value	10	9	7	4	∞	∞	6	0
Parent	B	C	D	H	Null	Null	H	-1

(a) Iteration 1:

$$\begin{aligned}
 S &= \{H\} \Rightarrow \text{Set of edges to look at: } \{\langle H, G \rangle, \langle H, D \rangle\} \\
 d'(\langle H, G \rangle) &= d[H] + c(\langle H, G \rangle) \Rightarrow d'(\langle H, G \rangle) = 0 + 6 = 6 \\
 d'(\langle H, D \rangle) &= d[H] + c(\langle H, D \rangle) \Rightarrow d'(\langle H, D \rangle) = 0 + 4 = 4 \\
 \min\{d'(\langle H, G \rangle), d'(\langle H, D \rangle)\} &= \min\{6, 4\} = 4 \\
 \Rightarrow S &= \{H, D\}, \quad d[D] = 4, \quad D.\text{parent} = H, \quad d[G] = 6, \quad G.\text{parent} = H
 \end{aligned}$$

(b) Iteration 2:

$$\begin{aligned}
S = \{H, D\} &\Rightarrow \text{Set of edges to look at: } \{\langle H, G \rangle, \langle D, C \rangle\} \\
d'(\langle H, G \rangle) = d[H] + c(\langle H, G \rangle) &\Rightarrow d'(\langle H, G \rangle) = 0 + 6 = 6 \\
d'(\langle D, C \rangle) = d[D] + c(\langle D, C \rangle) &\Rightarrow d'(\langle D, C \rangle) = 4 + 3 = 7 \\
\min\{d'(\langle H, G \rangle), d'(\langle D, C \rangle)\} &= \min\{6, 7\} = 6 \\
\Rightarrow S = \{H, D, G\}, \quad d[G] = 6, \quad G.\text{parent} = H, \quad d[C] = 7, \quad C.\text{parent} = D
\end{aligned}$$

(c) Iteration 3:

$$\begin{aligned}
S = \{H, D, G\} &\Rightarrow \text{Set of edges to look at: } \{\langle G, C \rangle, \langle G, B \rangle, \langle D, C \rangle\} \\
d'(\langle G, C \rangle) = d[G] + c(\langle G, C \rangle) &\Rightarrow d'(\langle G, C \rangle) = 6 + 2 = 8 \\
d'(\langle G, B \rangle) = d[G] + c(\langle G, B \rangle) &\Rightarrow d'(\langle G, B \rangle) = 6 + 6 = 12 \\
d'(\langle D, C \rangle) = d[D] + c(\langle D, C \rangle) &\Rightarrow d'(\langle D, C \rangle) = 4 + 3 = 7 \\
\min\{d'(\langle G, C \rangle), d'(\langle G, B \rangle), d'(\langle D, C \rangle)\} &= \min\{8, 12, 7\} = 7 \\
\Rightarrow S = \{H, D, G, C\}, \quad d[C] = 7, \quad C.\text{parent} = D, \quad d[B] = 12, \quad B.\text{parent} = G
\end{aligned}$$

(d) Iteration 4:

$$\begin{aligned}
S = \{H, D, G, C\} &\Rightarrow \text{Set of edges to look at: } \{\langle G, B \rangle, \langle C, B \rangle\} \\
d'(\langle G, B \rangle) = d[G] + c(\langle G, B \rangle) &\Rightarrow d'(\langle G, B \rangle) = 6 + 6 = 12 \\
d'(\langle C, B \rangle) = d[C] + c(\langle C, B \rangle) &\Rightarrow d'(\langle C, B \rangle) = 7 + 2 = 9 \\
\min\{d'(\langle G, B \rangle), d'(\langle C, B \rangle)\} &= \min\{12, 9\} = 9 \\
\Rightarrow S = \{H, D, G, C, B\}, \quad d[B] = 9, \quad B.\text{parent} = C
\end{aligned}$$

(e) Iteration 5:

$$\begin{aligned}
S = \{H, D, G, C, B\} &\Rightarrow \text{Set of edges to look at: } \{\langle B, A \rangle\} \\
d'(\langle B, A \rangle) = d[B] + c(\langle B, A \rangle) &\Rightarrow d'(\langle B, A \rangle) = 9 + 1 = 10 \\
\min\{d'(\langle B, A \rangle)\} &= \min\{10\} = 10 \\
\Rightarrow S = \{H, D, G, C, B, A\}, \quad d[A] = 10, \quad A.\text{parent} = B
\end{aligned}$$

(f) Iteration 6:

$$\begin{aligned}
S = \{H, D, G, C, B, A\} &\Rightarrow \text{Set of edges to look at: } \{\} \\
\Rightarrow S = \{H, D, G, C, B, A\}, \quad d[E] = \infty, \quad E.\text{parent} = \text{Null}, \\
d[F] = \infty, \quad F.\text{parent} = \text{Null}
\end{aligned}$$

(g) Iteration 7:

$$\begin{aligned}
S = \{H, D, G, C, B, A\} &\Rightarrow \text{Set of edges to look at: } \{\} \\
\Rightarrow S = \{H, D, G, C, B, A\}, \quad d[E] = \infty, \quad E.\text{parent} = \text{Null}, \\
d[F] = \infty, \quad F.\text{parent} = \text{Null}
\end{aligned}$$

(h) Iteration 8:

$$\begin{aligned}
S = \{H, D, G, C, B, A\} &\Rightarrow \text{Set of edges to look at: } \{\} \\
\Rightarrow S = \{H, D, G, C, B, A\}, & \quad d[E] = \infty, \quad E.\text{parent} = \text{Null}, \\
d[F] = \infty, & \quad F.\text{parent} = \text{Null}
\end{aligned}$$

4. (10 pts) Prove the following claim: for any cycle in a weighted graph with distinct **non-negative** edge weights, the edge with the highest weight in the cycle does not belong to an MST.

Answer

To prove the statement, we take the "Exchange Argument" approach.

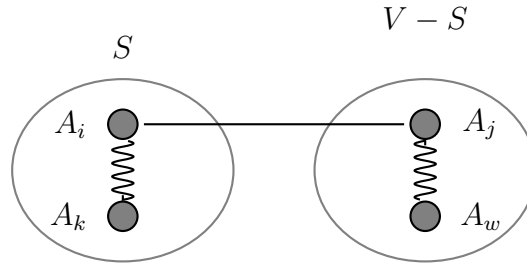
Proof by Contradiction

Suppose we have a graph $\mathcal{G} = (V, E)$ which contains a cycle $C = A_1 \rightarrow A_2 \rightarrow A_3 \rightarrow \dots \rightarrow A_{m-1} \rightarrow A_m \rightarrow A_1$ with distinct non-negative edge weights, where $\langle A_i, A_j \rangle$ has the highest weight. In addition, consider an MST T for $\mathcal{G}$, which contains $\langle A_i, A_j \rangle$. Since T is a tree and it doesn't have any cycle, there must be at least one edge in C that it's not in T in which its weights are less than $\langle A_i, A_j \rangle$. Suppose this edge is $\langle A_k, A_w \rangle$. Hence:

$$c(\langle A_k, A_w \rangle) < c(\langle A_i, A_j \rangle).$$

Now, consider $S \subset V$ where:

$$A_i, A_k \in S \quad \text{and} \quad A_j, A_w \in V - S$$



Since this tree doesn't contain $\langle A_k, A_w \rangle$ and in a tree there's a path between any two vertices, hence: 1) there's a path between A_i and A_k , 2) there's a path between A_j and A_w and 3) there's a path between A_k and A_w . One of the paths between A_k and A_w is the following:

$$A_k \quad \underline{\text{path}} \quad A_i \quad \underline{\text{edge}} \quad A_j \quad \underline{\text{path}} \quad A_w$$

We will be constructing a new tree T' by removing $\langle A_i, A_j \rangle$ and adding $\langle A_k, A_w \rangle$ and showing that T' is also a spanning tree.

For any two vertices $x, y \in V$ there will be three possibilities: 1) $x \in S, y \in V - S$, 2) $x \in S, y \in S$, 3) $x \in V - S, y \in V - S$. Since T is a spanning tree, there's a path from x to y . Now, there will be two cases:

- The path from x to y doesn't use the edge $\langle A_i, A_j \rangle$. Thus, there must also exist a path from x to y in T' .
- The path from x to y passes through $\langle A_i, A_j \rangle$. Thus, suppose the following occurs:

$$x \quad \underline{\text{path}} \quad A_i \quad \underline{\text{edge}} \quad A_j \quad \underline{\text{path}} \quad y$$

We can substitute this path in T with the following in T' :

$$x \quad \underline{\text{path}} \quad A_i \quad \underline{\text{path}} \quad A_k \quad \underline{\text{edge}} \quad A_j \quad \underline{\text{path}} \quad A_x \quad \underline{\text{path}} \quad y$$

Hence, T' is a spanning tree.

We know:

$$w(T') = w(T) + c(\langle A_k, A_w \rangle) - c(\langle A_i, A_j \rangle)$$

Since $c(\langle A_k, A_w \rangle) < c(\langle A_i, A_j \rangle)$, the term $c(\langle A_k, A_w \rangle) - c(\langle A_i, A_j \rangle)$ is negative, and:

$$w(T') < w(T)$$

This means we found a spanning tree T' which has less weight than an MST T , leading to a contradiction.

5. (20 pts) Given two strings, write an algorithm for finding the longest common subsequence. A subsequence of a string is a sequence of characters that conforms to the relative ordering of the characters in the string. The characters do not necessarily appear contiguously in the string.

Example: GHTCCHT and CHTGCH. The longest common subsequence is HTCH.

Design an algorithm using a Dynamic Programming strategy which consists of the following:

- (a) (7 pts) Formalize a recursive definition for the solution.

Answer: Let $S_1[1..m]$ and $S_2[1..n]$ be the two strings with length m and n and let $\text{LCS}(i, j)$ represent the length of the longest common subsequence (LCS) of the first i characters of string S_1 and the first j characters of sequence S_2 . We will show that the recurrence relation for LCS is as follows:

$$\text{LCS}(i, j) = \begin{cases} \text{LCS}(i-1, j-1) + 1 & \text{if } S_1[i] = S_2[j] \\ \max(\text{LCS}(i-1, j), \text{LCS}(i, j-1)) & \text{if } S_1[i] \neq S_2[j] \end{cases}$$

1 The base case: the base case for the recursive characterization corresponds to the case when at least one of the strings is empty (i.e., when $i = 0$ or $j = 0$), then $\text{LCS}(0, j) = 0 \quad \forall j$ or $\text{LCS}(i, 0) = 0 \quad \forall i$.

2 Recurrence:

- i. If $S_1[i] = S_2[j]$, then the LCS of $S_1[1...i]$ and $S_2[1...j]$ is 1 plus the LCS of $S_1[1...i-1]$ and $S_2[1...j-1]$, hence:

$$\text{LCS}(i, j) = \text{LCS}(i-1, j-1) + 1$$

- ii. If $S_1[i] \neq S_2[j]$, then the LCS is the maximum of the LCSs obtained by either excluding the last character of S_1 or S_2 , hence:

$$\text{LCS}(i, j) = \max(\text{LCS}(i-1, j), \text{LCS}(i, j-1))$$

Finally, the value of $\text{LCS}(m, n)$ gives the length of the longest common subsequence of the entire sequences $S_1[1...m]$ and $S_2[1...n]$. As an example, applying this recursive call to the given two strings of **GHTCCHT** and **CHTGCH** will be as follows:

		<i>C H T G C H</i>						
<i>Index</i>		0	1	2	3	4	5	6
	0	0	0	0	0	0	0	0
<i>G</i>	1	0	0	0	0	0	1	1
<i>H</i>	2	0	0	0	1	1	1	2
<i>T</i>	3	0	0	0	1	2	2	2
<i>C</i>	4	0	0	1	1	2	2	3
<i>C</i>	5	0	0	1	1	2	2	3
<i>H</i>	6	0	0	1	2	2	2	3
<i>T</i>	7	0	0	1	2	3	3	4

The value in $\text{LCS}(7, 6) = 4$ gives the length of the longest common subsequence of these two strings.

- (b) (10 pts) Write pseudocode for an iterative dynamic programming algorithm with a dictionary to implement the solution.

Answer:

Description: based on the above observation, we can write an iterative algorithm using a dictionary as follows:

$$\text{dict}[i][j] = \begin{cases} \text{dict}[i-1][j-1] + 1 & \text{if } S_1[i] = S_2[j] \\ \max(\text{dict}[i-1][j], \text{dict}[i][j-1]) & \text{if } S_1[i] \neq S_2[j] \\ 0 & \text{if } i = 0 \quad \text{or} \quad j = 0 \end{cases}$$

Finally, we need to backtrack through the **dict** table to find the actual longest common subsequence based on the following procedure:

We will start from the bottom-right corner of the **dict**, i.e., **dict**[m][n].

- (1) If the characters $S_1[i-1]$ and $S_2[j-1]$ are equal, this means this character is part of the LCS. WE Will add it to the result and move diagonally up-left (i.e., $i-1, j-1$).
- (2) If the characters are not equal, we will move in the direction that has the larger value:
 - (2.1) If $\text{dict}[i-1][j] \geq \text{dict}[i][j-1]$, move up (i.e., $i-1, j$).
 - (2.2) If $\text{dict}[i][j-1] > \text{dict}[i-1][j]$, move left (i.e., $i, j-1$).
- (3) Finally, we will reverse the founded LCS since it's constructed backward.

Pseudocode:

```

INPUT S1, S2,
dict[k][w] = 0 // stores the length of the LSC of two strings
with length k and w.
// Initialization for the base case
for i in range(m+1):
    dict[i][0] = 0
for j in range(n+1):
    dict[0][j] = 0

// Filling out the Dictionary
for i in range(1, m+1):
    for j in range(1, n+1):
        if S1[i-1] == S2[j-1]:
            dict[i][j] = dict[i-1][j-1] + 1
        else:
            dict[i][j] = max(dict[i-1][j], dict[i][j-1])

// Initialize an empty string to store the LCS
LCS = ""

// Start from the bottom-right corner of the table
i = m
j = n

// Backtrack to find the LCS
while i > 0 and j > 0:
    // If characters match, add to the LCS
    if S1[i-1] == S2[j-1]:
        lcs += S1[i-1]
        i -= 1
        j -= 1
    // Move up if dict[i-1][j] is larger
    elif dict[i-1][j] > dict[i][j-1]:

```

```

        i -= 1
    // Move left if dict[i][j-1] is larger
    else:
        j -= 1

    // The LCS is obtained by reversing the string
    LCS = LCS[::-1]
    return LCS

```

(c) (3 pts) Analyze the runtime of your algorithm.

Answer: This code has four main parts: 1) Initialization, 2) Filling out the dictionary, 3) Backtracking, and 4) Reversing. We will analyze the runtime of each chunk of code individually and figure out the final runtime:

- 1) **Initialization:** The first loop contains an atomic operation that takes $\mathcal{O}(1)$. This loop iterates m times. The second loop contains an atomic operation that takes $\mathcal{O}(1)$. This loop iterates n times. Hence:

$$\text{Runtime of Initialization: } m \times \mathcal{O}(1) + n \times \mathcal{O}(1) = \mathcal{O}(m) + \mathcal{O}(n)$$

- 2) **Filling out the Dictionary:** This part involves filling a 2D table ('dict') of size $m \times n$, where each cell represents a subproblem. The nested loops iterate through each cell in the table once. Since each iteration involves a constant amount of work (a comparison or a max operation), the total runtime for this part is:

$$\text{Runtime of Filling out the Dictionary: } \mathcal{O}(m \times n)$$

- 3) **Backtracking:** In the worst case, we may have to traverse through all the entries in the dictionary starting from the bottom-right corner (i.e., from m, n down to $0, 0$). This requires moving at most $m + n$ steps. Hence, the worst-case runtime for this part is:

$$\text{Runtime of Backtracking: } \mathcal{O}(m + n)$$

- 4) **Reversing:** Reversing the string is an operation that processes each character in the LCS string once. In the worst case, the length of the LCS can be the minimum of the lengths of the two strings, i.e., $\min(m, n)$. Therefore, the runtime of reversing the string is:

$$\text{Runtime of Reversing: } \mathcal{O}(\min(m, n))$$

Final Runtime: Combining all these parts, we get the overall runtime of the algorithm as:

$$\mathcal{O}(m) + \mathcal{O}(n) + \mathcal{O}(m \times n) + \mathcal{O}(m + n) + \mathcal{O}(\min(m, n)) = \mathcal{O}(m \times n)$$

The dominant term is $\mathcal{O}(m \times n)$, so the overall time complexity of the algorithm is $\mathcal{O}(m \times n)$.

List of Collaborators Ryan Holt and I discussed our solutions to question 1 in this homework. Additionally, I used some online resources to solve question 5.

In question 1, we briefly discussed our ideas about how the graph should be constructed. After this discussion, I realized I hadn't considered the second source of information and how its validity has to be checked. I double-checked the way I constructed the graph with Ryan and added a few lines to my code without discussing the details with Ryan.

While construing the graph in question 1, I needed a code that would tell me whether a graph is acyclic or not. However, it was nowhere to be found. We previously had the concept of back edge, and how having a back edge means the graph has a cycle, but to write the code, I also seek some help from this [Link 1](#).

In question 5, I still had a hard time finding the actual subset. In class, we typically cared about the sum or length rather than providing an example. Hence, I got some help from the two following websites [Link 2](#), and [Link 3](#). In particular, I learned how to use the dictionary constructed to find the longest subsequence and how the string we find needs to be traversed from these two websites. However, writing the actual code and doing the run-time analysis was all mine.